

may have some success. If your company provides this type of service, work with the product teams to determine the best place for testers to add value. Perhaps you can embed testers in the client’s product teams.

Often the testing service companies offer UAT or system integration testing. They can add value if they build good communication channels between the product team and the outsourced test team. For example, increments of features could be passed for testing while coding is still in progress, rather than waiting until the whole release is ready to test. Figure 19-5 shows one possible way to deliver to a system integration test team at the end of every iteration, and then to perform UAT when a feature is completed. This requires that all product teams work together to produce testable features for frequent delivery.

Testing-as-a-service organizations working with agile teams need a different relationship with agile development teams than with teams that use phased and gated development. They need new ways to specify contracts and must work harder to interact with the client development teams. See the links in the bibliography for Part VII, “What Is Your Context?,” for options you can explore for agile contracts.

If your agile team is using offshore or near-shore test teams, it’s a good idea to have at least a few testers colocated with the development team to represent the offshore testing team. They can attend the planning

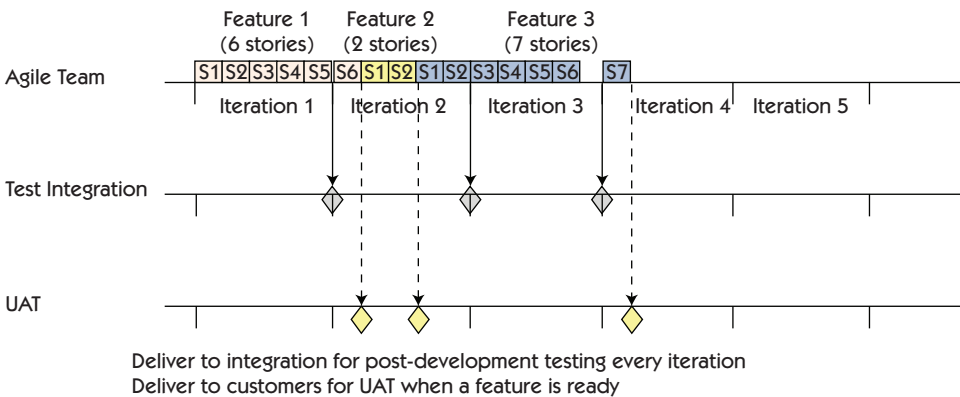


Figure 19-5 Testing as a service for integration testing and UAT

meetings and daily standups, pair with programmers, and keep the off-shore team in the loop. Recognize that while an offshore scenario may have perceived benefits for the business, it requires a lot more work and discipline to maintain sufficient communication and collaboration.

Working as an Offshore Testing Team

Parimala Hariprasad, who blogs at www.curious tester.com, discusses challenges and solutions she's found working on offshore testing teams.

I have served many teams that handle testing remotely. Some of these are startups scattered all over the world with limited resources and infrastructure. These small startups often collaborate with other developers, designers, and infrastructure providers to implement their business ideas. This means that my team is often acting as an offshore test team for multiple groups at once.

Introduction and Project Context

When any testing project starts, we get introductions in emails. We send a peppy five-liner about each tester on our team in the email with a funny picture. This sets an informal environment to start off. The email is then followed by audio calls for daily standups. Demos are usually done using Skype, Google Hangouts, WebEx, or other video-conferencing tools; we have explored a good mix of open-source and paid tools. In my experience, showing up face-to-face has worked best for my team because when someone speaks, putting a face to that voice gives a personal edge. Face-to-face doesn't necessarily mean flying your team to the main location. It could be facilitated by sharing pictures of each other, video calls, fun videos of a typical day at work, and so forth.

We get into the project context, explore the product under test, get additional information, ask questions, and start to work on the test strategy. I state early during initial conversations that work progress will be slow for the first couple of days as we are learning and exploring. We also share our deliverables with stakeholders 24/7 using tools like Dropbox and Google Docs so they can see who is working on a particular task at any point in time and what changes have been made since the previous day. This provides great visibility to our stakeholders at main locations.



Working Protocol and Ownership

We reiterate the working protocol that was agreed upon during the negotiation stages of the project (oh, yeah, our team does that). We talk openly about work timing in same/different time zones, testers who can provide backup (shadow testers), service-level agreements (SLAs) for critical issues, and so forth. We set common rules that go in as requests for collaboration, daily meeting times, test reporting format and processes, infrastructure availability, and necessary approvals to get started with work from day one. With some clients, we share samples of our previous work (masking confidential information) to help them get a better understanding of what value we bring to the table as a remote test team.

In distributed teams, the biggest challenge is lack of ownership between teams in remote and main locations. When things go wrong, some team members resort to blame games. In addition to skills, attitude plays a key role in how people take ownership of their work. Many teams I worked with had a mindset where everyone felt responsible for the team's success or failure. They constantly huddled to collaborate with all the teams with a common end goal in mind. They had a vision for the product they were testing. They didn't see testing as an isolated process, but as a key enabler in helping build great products.

Fun and Family

One of our team members hadn't seen one of the cofounders of a startup, although she interacted with him on a daily basis. A new idea was born. We shot funny pictures and video of our team at our cubicles and shared them with team members at the main location. They responded with a big smile and reciprocated this action—talk about a butterfly effect! We also sent flowers and gifts to our clients and their team members on special occasions. If we learned about their travel plans to our location, we hosted them at our expense, took them out for dinner, and discussed work over food. This has helped us to build strong bonds with our teams in main locations.

The latest videoconferencing technology also gives us the luxury of interacting with people in an informal setup where it's face-to-face—well, almost! Not many organizations can afford traveling to client locations. Low-cost videoconferencing tools can reduce travel costs. In some cases, some of my teams were airlifted to main locations after the clients saw the value we added to their work. This was a result of the credibility we built with them while working remotely.

Visibility and Feedback

Trust and credibility play a major role in defining how work gets done between the main location and the offshore testing teams. In the team

on which I worked, I provided great visibility into our day-to-day work, which included daily status reports, daily standups, weekly calls, separate emails for critical issues, having backup for testers going on leave or long breaks, and so forth. We didn't need a camera fitted to our desk to do that. We were very transparent. Seeking feedback during project retrospectives and responding to feedback are other key actions that will build trust and credibility.

In the next sidebar, “Will the Real Manager Please Stand Up?,” Parimala shares a few more challenges that are more conflict-based: working with managers who have different goals, or managers who only know how to say yes. She explains how she worked with her team to solve some of the conflicts.

Will the Real Manager Please Stand Up?

In one distributed team I worked for, my testing team reported to a line manager who handled operations like procuring systems, approving leave requests, doing performance appraisals, and determining compensation at the remote location. Technically, my team reported to a technical manager who sat in the main location with his team there. Many decisions used to be pushed onto our team with hardly any discussion or information. Many times, we had to blindly do what they asked us to do. We were not supposed to say no. We were not supposed to ask questions. At least, that's what our line manager told us. The way we dealt with it was by building credibility about my team's contribution to the project. Over a period of time, team dynamics between us changed for the better. Sometimes, you may need to work harder to create a healthy working culture between teams.

In some projects, it is common that the line manager and the main location manager have a different understanding of what's important. In such cases, the teams suffer because they don't know whom to listen to and whom to please. If you keep the line manager happy, the other manager is going to shout from the rooftop about poor deliverables. If you keep the other manager happy, your performance appraisals are going to be affected—this is how the testers are threatened at times. This leads many testers to try to please both of them as much as possible by keeping both of them in the dark. Such office politics end up hurting team morale and eventually disintegrating the team.

An extreme case is when offshore testing teams end up saying yes to every demand that comes from the main location whether it is reasonable or not. I have personally worked on a couple of projects where offshore testing teams overcommitted without keeping delivery challenges in mind. This in turn brings stress due to unrealistic expectations.

Handling Conflicts of Interest (When Gods Go Crazy)

Any team's true strength and character are revealed when conflicts arise. Sometimes, conversations around conflicts go on forever. The following methods have helped the teams I work with immensely:

- **Switch the media:** If a topic is bounced back and forth more than three times in an email, it's time to pick up the phone or call for a meeting.
- **24-hour rule:** It's OK to write a heated email; just wait until you calm down before you send it, perhaps 24 hours.
- **Face-to-face rule:** Initiating difficult conversations is one of the most challenging things to do in a conflict-filled environment. A wrong start can have devastating consequences. I coach my team to initiate difficult conversations with empathy, face-to-face (on video or in person). We start with stating the problem and the context in which it occurs, explain what attempts we have made to resolve the problem, and maintain a positive resolve. In cases where it is already known that two individuals have conflicts, we bring in a common colleague/positive influence to take charge and help resolve the conflict.

In my experience, skills, language barriers, and time zones have been small challenges. Passion, attitude, courage, and excellence are more difficult to learn yet necessary for working in harmony in a distributed setup.

Parimala brings up issues that teams may not anticipate, such as conflicting messages from managers in different locations and a lack of a feeling of ownership by teams in different locations. You can't anticipate every problem, but having some guidelines such as Parimala's rules about switching media when the current communication style isn't working, or having difficult conversations face-to-face, will help you deal with issues as they occur.

TOOL IDEAS FOR DISTRIBUTED TEAMS

Throughout this chapter, we've mentioned tools that may help you solve a particular problem. Here's a summary of what we think might help you. If you think a particular tool will work in your team's process, experiment with it for an iteration or two. The tool should fit the process, not the other way around, so don't retrofit your process to fit the tool. These need to be simple techniques, such as starting an impromptu "Three Amigos" conversation to discuss requirements for a story or pairing for exploratory testing. If the learning curve is too steep or the practices are too complicated, they won't be adopted long-term.

Communication Tools

Use **email** for broadcasting information that is one-way, such as follow-ups on individual conversations. Since emails can easily be misinterpreted or misunderstood, try reviewing emails on sensitive topics with someone else before pushing the Send button.

Use physical or virtual task/story/kanban **boards** to track progress and make testing activities visible. Try the simplest solutions first. If it's enough to post a photo of the kanban board on the wiki twice a day, do it. If an online tracking tool is more appropriate, try more than one to see which fits best. Some tracking tools aren't designed to accommodate testing tasks, but you can usually find a creative solution.

Collaboration Tools

Phone, Voice-over-Internet Protocol (VoIP), and **videoconference** tools enable two or more people to collaborate and share ideas. Experiment to see which tools work best for you since each one seems to have its own problems. Use something as simple as Skype, Google Hangouts, or Zoom to have a three-way conversation can prevent many misunderstandings, such as how a feature should behave or what browser versions are supported.

Be aware of language issues, and remember to speak clearly and slowly if participants on the call don't share a common first language. Use headsets when appropriate to keep your hands free to type, make notes, or

look up information, but if you're in a group meeting, pay attention to the conversation instead of multitasking. Use **video** if bandwidth permits; it allows people to connect on a much more personal basis. This means that all team members need good-quality webcams and microphones. Think about the story of Lisa's virtual telepresence device in the section on strategies.

Lisa's Story

When I joined my current team, they had been using Skype voice-only for daily standups. We started using the video feature of Skype. This helped everyone feel more connected, and the visual communication added value. However, each day, at least one participant's call would get dropped, or the audio would echo or start "roboting," meaning that voices sounded like a mechanical robot talking.

We decided to try Google Hangouts for standups and meetings. At first this was a big improvement. But after a few months, it started to exhibit similar problems to what we'd experienced with Skype.

Our next experiment for standups and meetings was Zoom. So far, this has been the most reliable solution, though sessions can freeze up and audio can echo.

Our programmers pair all day, and since we're distributed, this means that there are always a few remote pairs. They also use Skype, Google Hangouts, or Zoom for pairing. These tools usually work better when there are only two participants, but they do seem to go through cycles of problems when upgraded to new versions. We also sometimes experience bandwidth issues. We get help from our IT department, discuss the issues in our retrospectives, and continue to experiment with videoconference tools.

For some situations, **text chat** is good enough. Two or more people can easily share ideas, especially if they type fast! We have written much of this book via Google Talk, asking quick questions when we are writing. Text chat allows you more time to think about what to say. It also lets you ping someone who might be in the middle of something now but can get back to you in a few minutes. It works well for questions that are not urgent. If the discussion gets more complicated, you can jump on videoconference.

As mentioned earlier, a team **group chat** is an effective way for a distributed team to communicate in an unintrusive way. Group chat software can be set up with alert sounds, so that if someone types a name, that person will know to check the chat. This allows the team to be heads-down on work but still seek and give help as needed.

Wikis are widely used in many companies, while others use other collaboration tools such as SharePoint. These are good ways to record decisions and share information across multiple locations, but be sure they are kept well organized and easily searchable.

Mind mapping is a powerful collaboration tool for many testing-related activities such as test planning, brainstorming test scenarios, and tracking results. There are many mind-mapping products available that support real-time collaboration. Experiment with different offerings to see which works best with your team's infrastructure.

To pair test or code remotely, you need **desktop sharing** so that each person can see and control the screen as desired. You can use screen sharing built into your operating system, or try a variety of solutions such as a virtual private network (VPN), Skype, and Office Communicator. Obviously, make sure that whatever you use is secure.

As we noted in *Agile Testing* (pp. 82–83), **defect-tracking systems** (DTSS) are not a good communication tool, and debate in the agile community continues as to whether bugs should be logged or simply fixed and documented with an automated test. Experiment with different approaches to find what works best for your team. Revisit your defect-handling process periodically to see if what you're doing still fits.

Short feedback loops are the heartbeat of an agile team. We need reliable infrastructure to support that quick feedback. CI, developer sandboxes, test servers, databases, shared drives, virtual machines, test labs, and other infrastructure are critical to any team but, like everything else, even more of a challenge for distributed teams. A green CI ensures that the team halfway around the world can be productive as soon as they get to work. Hardware and software needed by team members in various locations must be reliable. Remote team members need the capability to solve their own infrastructure issues or have help available at all times.

Don't forget to include your system administrators and DevOps practitioners when teams from different locations gather together or hold team-building activities in their own locations. Make sure people in these roles in various locations stay connected with each other. They can collaborate to **build the infrastructure** that best supports the distributed teams (Hagberg, 2013).

If your team is distributed, you may have a stronger need to test your infrastructure. Teams are now testing their infrastructure and environments with automated regression tests, just as they test their production code. See Chapter 23, "Testing and DevOps," for more on infrastructure testing.

Use your team **retrospectives** to identify impediments to collaboration and communication in your distributed team. Try different brainstorming techniques such as impact mapping and mind mapping to set goals for improvement and think of different solutions you could try. We recommend trying two possible approaches side by side for easier comparison.

SUMMARY

Today's technology provides good ways for distributed team members to collaborate and communicate. However, your distributed team will need extra discipline and creativity to keep everyone engaged in testing before, during, and after coding. In this chapter, we discussed many ways distributed teams can succeed at building quality into their software and preventing defects.

- When talking about distributed teams, we include dispersed teams with remote members, teams with sub-teams in multiple locations, and offshore teams. All these situations can get in the way of whole-team responsibility for quality and testing activities.
- Be aware of language and cultural barriers. Help teams in different locations feel safe and free to ask questions. Give them time to make sure they understand what's been said.

-
- Get everyone together in person often. When that's not possible, use video meetings, and try some fun activities to help the team members build their comfort and trust levels.
 - If possible, have managers, customers or their representatives, coaches, and leaders in different disciplines available at each location.
 - Tests written in a shared DSL help ensure that all development and customer team members have a common understanding of how features should behave.
 - Experiment with the communication and collaboration tools discussed in this chapter and seek out more if needed to find the ones that work best for your team.
 - Make use of CI and other infrastructure to keep code integrated and up-to-date, and feedback loops short.

This page intentionally left blank

AGILE TESTING FOR MOBILE AND EMBEDDED SYSTEMS

20. Agile Testing for Mobile and Embedded System

Similar, Yet Different

Testing Is Critical

Agile Approaches

Humans have been embedding software in devices and machines for decades. Early on, this was the province of rocket science. The Apollo Guidance Computer, developed for the Apollo space program, is one of the earliest examples (Wikipedia, 2014c). The use of embedded software spread as microprocessors and microcontrollers became less expensive and more widely available.

What do we mean when we use terms like *mobile app* and *embedded software*? We like the definition Jon Hagar uses (Hagar, 2014):

Mobile software can be found in smartphones, automobiles, and other devices that move. Typically, mobile systems have Wi-Fi and/or cellular network connections, are powered by batteries, have large numbers of hardware platforms, have various resource constraints such as screen size, limited user inputs, and smaller amounts of memory, storage space, and CPU speed than a laptop or desktop computer.

An embedded device is an electronic device that contains software, yet the users may not fully appreciate that they are using software due to unique hardware and a limited user interface. Examples of embedded software devices include braking systems in cars, control systems in airplanes, medical devices such as pacemakers, and “smart” light switches. Mobile and embedded software programming introduces unique programming and testing challenges compared to IT systems.

Proprietary embedded devices such as heart monitors and health and fitness equipment have similar testing challenges. Many of these embedded software products and proprietary devices are in regulated industries such as health care and aerospace. We will focus on agile testing in regulated environments in Chapter 21, “Agile Testing in Regulated Environments.”

Agility is especially important in a fast-changing market such as mobile apps, and agile development is a natural fit for teams in that space. With the explosion in the number and types of mobile devices, from smartphones to “wearable” computers, more agile teams are faced with the challenge of developing embedded and mobile apps.

In Chapter 12 of *Agile Testing*, we described an example of testing a system that used embedded software. We’ve gathered more examples here to illustrate how agile testing values, principles, and practices help teams that deliver embedded and mobile products, with more focus on mobile.

SIMILAR, YET DIFFERENT

Agile testing values, principles, and practices apply to mobile and embedded software just as they do to any other types of software applications. Short feedback loops, collaborating with the customer, the simplest thing that could possibly work, test-driven development (TDD), guiding development with tests, and a whole-team commitment to quality all apply to testing software that runs on mobile devices or as part of a larger software solution.

There are definitely differences, though. As Jonathan Kohl notes (Kohl, 2013), we are unlikely to pick up our monitors or laptops and shake them. We’re not going to rub our hands all over their screens, although that is changing. For example, Janet just purchased a new laptop with Windows 8.1 and used the touchscreen to scroll while editing. Certainly we’ll use our laptops at the local coffee shop, but we’re less likely to use them while riding the subway or hiking in the mountains. We don’t switch continually from portrait to landscape orientation on laptops, but we do these things all the time with our mobile devices, which are full of sensors, cameras, and other bells and whistles. Then there are all

the other components that allow mobile devices to work, such as cell phone towers, GPS satellites, and batteries.

The number and variety of the different mobile devices add yet another level of complexity to mobile app testing. Even the service provider can make a difference. Trying to test every permutation of platform and device approaches the impossible.

There are tools to help. We can take advantage of analytics software to see what users are doing when the software fails. If many users bail out of a mobile app on a certain view, we'll know about it, and we can try to figure out what usability issue might have caused it. It's about getting fast feedback, only this time it is from the users themselves.

The Ministry of Testing created a mind map of mobile testing ideas that you can see online (Sherry, 2013). Use it to generate your own ideas and to create your own mind map specific to your app. Figure 20-1 shows the data node of the mind map as an example.

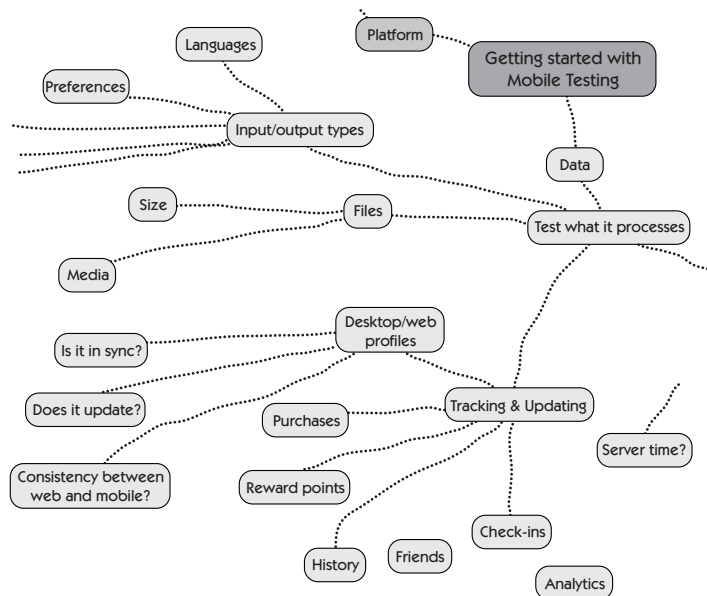


Figure 20-1 Ministry of Testing mind map for mobile apps

Embedded software products are more likely to be confined to one device and environment. The elliptical machine at your fitness center gets a lot of use, but it stays in one place, and its software executes in predefined ways. However, more and more consumer products, even automobiles, offer APIs that allow users to customize their own applications. As we use more software throughout our daily lives, building the right product for all customers gets more challenging.

TESTING IS CRITICAL

Testing has the potential to add even more value in embedded and mobile product development than in other software products. As Julian Harty has pointed out (Harty, 2015), if you release a new mobile app and get negative reviews right away, your app may never recover. If users aren't delighted by your product, it may have a one-star rating within a matter of minutes, which could be the end of your app before it's begun. It's difficult to fix problems fast enough to reverse the first impression. Your customer has many more choices that are easily available in the App Store. Just to get your product off the ground, you must be confident about all of its quality attributes. Validated learning techniques such as A/B testing, which we talked about in Chapter 13, "Other Types of Testing," may be an appropriate type of testing to use.

Some dimensions of quality may be more important in some than in other types of software products. For example, the ability to download an app via your mobile provider or to display the user interface (UI) on a small screen is important to verify when testing a smartphone, but not as critical on a tablet. The personal nature of mobile apps adds dimensions not seen in web-based or desktop software products. Julian Harty notes that since mobile apps may want access to your location, your contact list, your calendar, and other highly personal information, users need to feel they can trust the product.

Testers, programmers, and user experience (UX) designers can work together to ensure that end users feel safe and find the app valuable and easy enough to use. Peter Morville (Morville, 2004) describes different facets to consider for user experience as shown in Figure 20-2.

Testing mobile apps is, of course, different from verifying high-risk applications such as computers that guide space capsules and rockets

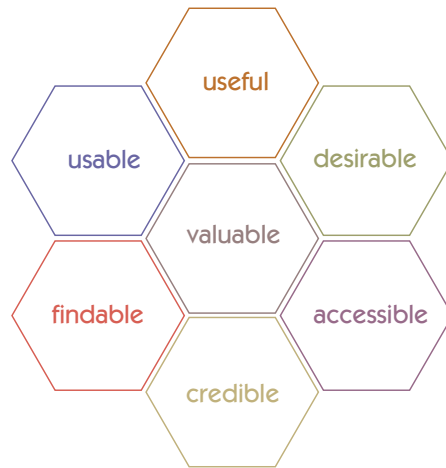


Figure 20-2 The user experience honeycomb by Semantic Studios

that use embedded software. Embedded software is also developed for safety-critical products such as medical devices, where testing outcomes may be a matter of life or death. In this context, it's more obvious that testing must be an integral part of software development. Teams must verify quality attributes such as reliability, stability, and tolerance of certain hardware changes.

AGILE APPROACHES

These products derive the same benefits from agile practices such as TDD, pair programming, continuous integration (CI), a whole-team approach to quality, and guiding development with customer-facing examples as in acceptance-test-driven development (ATDD)/specification by example (SBE)/behavior-driven development (BDD).

Lisa's Story

I jumped at the chance to help with testing our team's iOS app. I was an extremely late adopter of smartphones and tablets myself, and I couldn't even intuit the correct gestures for standard touchscreen functionality. Luckily, the iOS developers quickly got me up to speed with testing basics for physical devices, Xcode, and Crashlytics. They practice TDD and pair programming to develop our iOS code, which is supported by CI.

Testing a mobile app as a newbie is one way to find bugs and usability issues. I found that my general testing experience applies to iOS testing as well. As with any software product, we're trying to deliver the most valuable features for our users and our business. We also have to satisfy the App Store, which is a major stakeholder. Also, the mobile app supports our software-as-a-service (SaaS) web product, so it is helpful that I'm familiar with how it should work.

However, as I started learning more from conference sessions, articles, and books by mobile testing leaders such as Julian Harty, Karen N. Johnson, and Jonathan Kohl, I realized there's another huge level of complexity to mobile testing. I have to learn new skills and new ways to think to be more effective.

As I learn more, I'll be able to contribute more to our team, which has already started improving the design and architecture of our mobile app code. They're also learning new ways to automate more functional and regression testing.

We asked agile testing practitioners who are experienced in mobile app and embedded software development to share their insights into doing agile testing in these domains. We hope their stories will inspire experiments for you to try with your mobile or embedded teams.

Lessons Learned with Agile in Mobile and Embedded Domains

Jon Hagar, *the author of Software Test Attacks to Break Mobile and Embedded Devices (Hagar, 2013), tells about mobile and embedded teams using agile testing principles and practices.*

Once upon a time, embedded systems were typically created using phased and gated development methods, so agile practices were seen as a big change. This is a bit of a misconception. There were early and successful adopters of agile in the embedded domain. The Atlas rocket embedded software team (Waters, 2004) adopted some agile principles and practices over time. For example, they used CI with smaller, more frequent software builds; focused more on developer-level testing with automation (not quite TDD but close); allowed the system to unfold over time versus following rigid plans and specifications; ran

automated tests with an exploratory focus; and had direct user/customer involvement.

I've heard people say that agile is for software only, not for hardware-focused projects (as embedded and some mobile devices tend to be). Success stories provide the counterpoint to this generalization. For example, Wikispeed (Wikispeed, 2014) is largely a hardware project following agile ideals. Wikispeed uses Extreme Manufacturing, which involves agile design, Scrum, kanban flow optimizations, Extreme Programming practices, and other agile approaches to rapidly turn ideas such as comfortable commuter cars into products customers can use.

Parallel hardware and software development efforts mean gaps in functionality and integration issues are likely to happen. Agile test concepts with CI and fast feedback help this by providing information to developers. For example, in embedded systems, typically the hardware is being developed at the same time the software is being created. It is a constant "chicken and egg" problem. The hardware defines how aspects of the software are to be built, and the software needs to be in place to assess many of the features that the hardware provides. In the CI of embedded systems, it is common to build a little hardware, integrate with some software, change the hardware in a final design, and then at the end have a more complete definition needed to finish the software. CI makes this parallel evolution efficient.

It is also common in the embedded domain to have last-minute software changes to accommodate hardware problems. I hear statements such as, "Well, the hardware is fixed, and it is easier to change the software," which may be true, but it comes at the risk of last-minute bugs introduced into the software.

A similar situation exists in the mobile app world where the app may need to work with many different OS versions and/or hardware platforms. Here, the CI problem arises across these different configurations and leads to concerns about combinatorial testing and "will it work" for enough of the market to be a success.

Based on stories such as these, here are a few of the lessons I've learned in implementing agile in mobile and embedded domains:

- Many teams, especially those that develop mobile apps, believe that time to market overrides the need to do testing. However, if you look at online reviews and ratings, you will see that having many bugs lowers ratings, and thus the likelihood of an app being successful.
- Late changes in hardware impact the product even more strongly than changes in software, so testing must be very agile

to accommodate them. When hardware and software are developed in parallel, CI is especially important. Testers should expect last-minute changes in the software in order to solve hardware problems. Both automated and exploratory testing is needed. Look for patterns of errors in the code. Include attack testing to prevent bugs from escaping into production.

- Start testing as early as possible. Use test doubles such as mock objects and test stubs for components that aren't yet delivered. Simulators can also help give testing a head start.
- Test automation can be more difficult in mobile and embedded domains. Although they're changing, many older test tools do not work for mobile and embedded devices because of the unique hardware interaction. Because of this, testers in agile teams may need many more technical skills on top of basic test knowledge. For example, understanding hardware design concepts, code, and development tools may be needed for successful testing.

Do you recognize your team in any of Jon's lessons learned? Consider these hard-earned lessons to help your own team build quality into your mobile or embedded software product. The practices and techniques Jon describes aren't new; they may just be needed at different times and in different ways than for testing web applications or other products. You can adapt familiar patterns and strategies for effective mobile and embedded testing, as we'll learn in the next story.

Strategies for Test Automation in Mobile Testing

Jeff "Cheezy" Morgan, author of *Cucumber and Cheese*
(Morgan, 2013), explains how he tests as he develops mobile apps and accommodates short release cycles.

I approach mobile testing in nearly the same way I approach testing a web application. I create my Screen Objects (Page Objects) for each screen in the application, define the data sets I will be using, build out my different navigation routes, and then construct the tests to leverage these artifacts. Although the structure of the code is nearly identical, there are a few things about testing mobile applications that are completely unique.

When testing web applications, you have to test against the various target browsers and versions of those browsers. You have the same thing with mobile applications, but it is more complicated. You have to concern yourself with different devices as well as different versions of those devices. In addition, you have to test your application on different versions of the operating system on all of those different devices. The matrix of targeted platforms can get quite large very quickly.

To make things even more complicated, there are different types of mobile applications. There are native applications, web applications, and applications called hybrids, which are a combination of the two. The tools or coding libraries you use to automate the tests might have to change depending on the type of application.

Finally, there are two major categories of tests that need to be performed on mobile applications: testing the behavior, and testing the appearance. The appearance is divided even further depending on the type of application. If it is a native application, you are really testing the layout of the controls that constitute the screen. If it is a web application, you are testing the responsive design of the web pages as displayed on the screen. I always build automation to test the behavior but rarely to test the appearance. I find that automating the visual aspects of the application takes a long time, and the tests tend to be brittle, causing a lot of false test failures. With all of this complexity, it might seem like an overwhelming task to figure out how to adequately test your application. Here's what I do.



The first thing to do is clearly understand your target devices. It is not difficult to get mobile usage reports on the Internet. (See the bibliography for Part VII, "What Is Your Context?," for links to examples of these.) I use these lists to create my own list of devices we plan to target. Don't forget that you need to make it a combination of devices and operating systems. In some cases the same device will appear more than once but with different versions of the operating system.

With that list ready, I acquire one or two of the top four devices. I connect one of each device directly to my build server so all of my automation runs continuously against the devices every time a developer checks code into the source code management system. If I have only one of each device, I also create an emulator for those devices on the build server and configure the build to first check for a device and run the tests on the emulator if it does not exist. I perform visual verification and exploratory testing against any device that is not connected to the build server.

The final thing I do is create several emulators to perform testing on devices that I do not have. I place these emulators on the build server

and create a build that runs nightly and runs the automation against each of these emulators. In addition, I create three emulators on the tester's computer that represent the large-, medium-, and small-screen resolutions of the targeted devices. These emulators on the tester's machine are used for visual verification of the application.

Cheezy combines automated tests and humans to verify the behavior and appearance of mobile apps. If you don't already have the type of infrastructure he describes, such as build servers and emulators, your team can address this together, perhaps with help from DevOps (see Chapter 23, "Testing and DevOps," for more on testing infrastructure).



In our experience, leveraging automation to make exploratory testing more effective is a winning strategy. The following story illustrates some aspects of this.

Automation-Assisted Mobile Exploratory Testing

Jon Hagar *explains how exploratory testing of mobile and embedded software can benefit from automation.*

Many practitioners think of exploratory testing as something that should be done manually by a thinking tester who plans, designs, executes, and then learns about the product interactively in real time. Automation can support exploratory testing of mobile and embedded systems without restricting the tester's freedom to explore different avenues. For example, automation is helpful in creating test inputs, especially for combinatorial testing, database population, and supporting test techniques such as boundary value analysis and equivalence classes.

Data capture and playback tools can assist in providing some repeatability during an exploratory test session. After testing, automation can help with analyzing test results. Automated test scripts can help with reviewing large log files or scanning for particular patterns, incorrect outputs, or negative trends. Bug reports and other test information can be generated automatically, providing good ways of visualizing the data. This information can be used for further analysis and manual exploration.

Above all, we need thinking testers, but test automation can help in certain contexts. The following examples illustrate situations where automation was invaluable for the exploratory testing effort.

My first story concerns an embedded medical system. Here the software under test could pose a risk to human life, so risk-based test planning with supporting tool analysis was needed throughout the project's development cycles. Testing focused on high-risk items first. The testers were also worried about tester bias, so they included the use of combinatorial test tools to provide mathematically sound input cases. The test inputs and outputs for the software-device under test were processed at a 10-millisecond rate. Consequently, a large real-time test simulation automation environment was needed to provide inputs every 10ms and record over 1MB of test data outputs generated during each test sequence. The large amounts of data generated and recorded also needed computer processing tools to locate possible bugs, look for trends, and visualize the test data results. Finally, the results were reported in online tools such that developers and user/customer stakeholders could review and analyze test results quickly and at their convenience. Each of these areas of automation required its own development effort to create the tools, but these added efforts were offset by better testing and a higher-quality embedded product.

My second example concerns a mobile app game under first-time development. Here, the developers wanted the ability to quickly repeat the actions the tester had done to find a problem. The testers used a capture-playback tool running on both a simulator and the actual mobile phone devices to capture the exploratory session. If a bug was found during the iteration, the captured session was electronically sent to the programmers, so they could replay it to support their debugging efforts. If no bugs were found, the capture script was kept only until the end of the next iteration, since changes were rapid and scripts would be out-of-date quickly. However, sometimes a script might be reused if changes to it were insignificant. Also, the testers used an online gaming checklist tool (see the bibliography for Part VII for suggested links) to help testers remember what to look for and test during an exploratory test session. This feedback with rapid semiautomated test execution produced the information that allowed a fun and functioning game to be produced. After the product was fielded, new releases used more advanced forms of test data generation and test case reporting to continuously improve the game. The initial time to market was quick with a very short final test phase, and later the product was continuously tested to improve it for players.

See Chapter 12, “Exploratory Testing,” for more ways to use automation to facilitate exploring. Get hardware and software team members together to look for automation opportunities. For example, use automation to record hardware and OS conditions on timing, temperature, or voltage while the tester invokes a series of user steps of software functionality.

Learning Agile Testing for Mobile Software

JeanAnn Harrison, a software tester from the Boston area in the United States, explains how working closely with programmers helped her get up to speed quickly.

My first introduction to mobile software testing, which was on the Los Angeles Police Department’s traffic ticket device, also gave me exposure to applying an agile methodology to a software project. Prior to this project, I had no formal agile training or mentoring. That first experience with testing mobile software was an immense learning experience.

Oh, sure, there were requirements to start, but most of our requirements were written while we were “feeling out” the software design. I realized how closely I needed to work with the developer, to be able to learn the intent of the design. At the same time, we were testing to see if the design made sense in terms of the overall set of requirements from the customer. We discussed scenarios such as the police officers needing to be able to store all their data from a day’s work on the device, and then upload that data once their shift was done. Timing and performance became critical to how the mobile software requirements unfolded.

Decisions on usage turned into design discussions based on exploratory testing. Quickly, I realized that with each sprint I needed to set some goals prior to testing. Planning what kinds of tests needed to be done within the scope of the project and schedule made my testing task much easier.

The experience turned out to be a relatively short project, but this is where I was first exposed to time-boxed iterations even though we didn’t give our process a name. I basically taught myself what would work based on how I interacted with the builds and the programmer. We did what worked for us, the project, and our users.



Taking on a new domain and development process is easier when you collaborate with other delivery team members. Mobile software testers have to go beyond the software to understand how the OS and hardware conditions affect software behavior. Continual collaboration among programmers, testers, UX designers, system administrators, and other specialists is vital for products to succeed in today's ultracompetitive market.

SUMMARY

Agile testing values, principles, and practices apply to mobile and embedded software development, but these domains present unique challenges. Extra attention has to be given to testing embedded software and proprietary mobile devices that operate in life-critical situations.

- Mobile devices and other devices with embedded software get used in ways that traditional computers don't, and these differences present unique challenges for testing.
- Device size and resource constraints require different approaches to test design.
- Testing may add even more value on mobile and embedded systems due to potentially high risks from both marketing and safety perspectives.
- Agile approaches work well for testing mobile apps and embedded software because of the fast feedback cycle, but there are unique obstacles such as limits to beta releases and slow store approval processes for iOS devices.
- Some automation may be needed for effective exploratory testing of software in mobile and other devices. Teams should collaborate to see what tooling can help.
- Experiment together to find testing approaches that are appropriate to the product, device, and delivery time frames.

This page intentionally left blank

AGILE TESTING IN REGULATED ENVIRONMENTS

21. Agile Testing in
Regulated Environments

The “Lack of Documentation” Myth
Agile and Compliance

Since the dawn of Extreme Programming, we’ve been told by some experts that agile development isn’t appropriate for safety-critical, highly regulated software, such as medical device and space flight software. Apparently a lot of teams didn’t get that memo. As you saw in Chapter 20, “Agile Testing for Mobile and Embedded Systems,” we’ve heard many success stories about teams in regulated domains such as aerospace and medical devices that have embraced agile values and principles. Many businesses must comply with financial regulations such as Sarbanes-Oxley (SOX). These companies have adapted agile development to fit their needs, and they’ve found how they can reduce risk with agile practices.

THE “LACK OF DOCUMENTATION” MYTH

Some articles explaining why agile is not appropriate for regulated domains cite lack of documentation. We continue to hear this fundamental misunderstanding even as the term *agile development* is well into its second decade. As we’ve mentioned earlier, agile development is about delivering value frequently, at a sustainable pace. There’s no inherent reason that software developed using agile values, principles, and practices would have inadequate documentation.

When we use an example-guided approach to software development, we turn examples into tests, and we can automate many of those tests. As David Evans and Gojko Adzic have noted (Adzic, 2011), the automated tests become living documentation. The beauty of documenting code

by way of automated tests is that since you have to keep regression tests passing, you are always keeping your documentation up-to-date. We gave an example of this in *Agile Testing* (p. 402). Accurate documentation is critical in domains where people's lives, security, or money is at stake.

AGILE AND COMPLIANCE

Regulatory agencies impose rules, and everyone governed by those rules must prove that they are complying with them. We must test to make sure that our software that tests compliance produces accurate results.

Lisa's Story

In my eight-plus years working on a team that developed a financial services application, I got lots of experience with verifying that our software complied with federal regulations—or that it helped our customers comply with those regulations.

Our customers used our software to manage employees' 401(k) retirement plans. Each year, they had to prove that they operated within IRS rules. For example, executives couldn't take advantage to pad their own accounts with tax-free savings. The rules were extremely complicated and often counterintuitive, but I enjoy learning challenging business domains.

Our product owner gave us many examples of sample inputs and expected outputs. Turning those into tests that guided coding ensured our success. We fearlessly delivered highly complex software that reliably automated compliance testing for our customers, so they weren't at risk for incurring fines.

Our competitors couldn't believe we were able to do this. But we had the tests to prove it!

Later, we were faced with "Bernie Madoff rules," regulations to help prevent brokers from scamming investors. These involved things like disclosing record-keeping fees and preventing plan advisers from changing investors' personal information. The software to comply with these rules didn't generate revenue, but failure to comply could mean significant fines for our company and our customers. For each regulatory requirement, we worked with the business stakeholders to find the simplest software solution, minimizing cost but ensuring compliance and security.

Audits and regulatory submissions impose accountability and other requirements needed to get a stamp of approval. Many of those

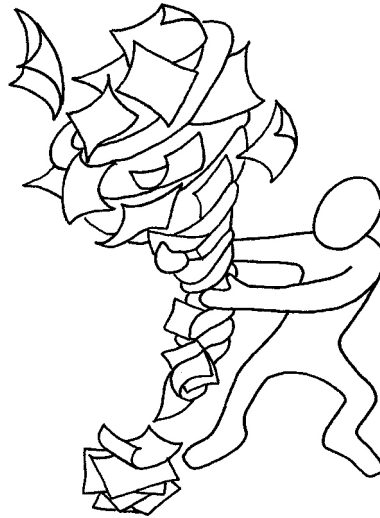


Figure 21-1 Wrestling with documentation

regulations say, “show evidence that” or something similar; unfortunately many organizations often interpret this to mean, “give me more documentation,” and they spend their time, not testing, but documenting (see Figure 21-1).

Janet's Story

A while ago, I was at a company that wanted to start doing more agile development and still needed to be SOX compliant. I looked at the company's standards and saw that they were anything but agile. The test strategy document had layer upon layer of required documentation, and the people who were tasked with writing it weren't even sure who was reading it, or why it was needed.

The agile teams were struggling to figure out what was absolutely necessary to fulfill their compliance but not get overwhelmed in documentation that had little meaning and did not satisfy their needs.

There are many companies that are able to comply with regulatory rules without excessive documentation (see Figure 21-2). Generally, as long as the company documents what it does, does what it documents, and

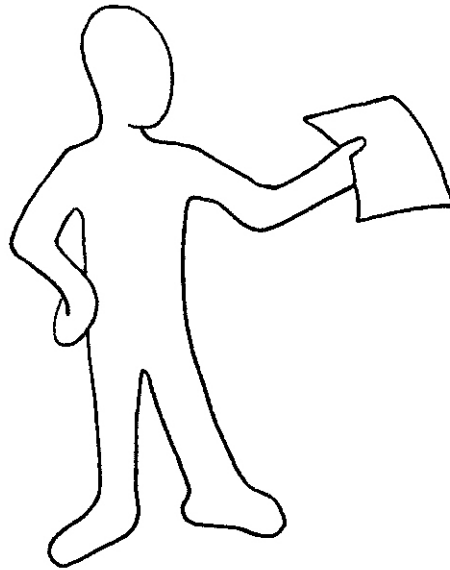


Figure 21-2 Simplify your documentation.

can produce evidence that it is doing what it says it is doing, regulators will be fine with that. It is worth your while to read the regulations and then take a pragmatic approach to meeting them. For example, taking photos of the story board on a regular basis and having those available for review may be a perfectly legitimate way to document what has happened for compliance purposes.

Make your auditors (they are stakeholders) part of the solution, and define their needs as stories or tasks so the deliverables are built into the solution. Consider tasks that need to be completed to comply with regulations when your team creates a definition of “done” for the next release. Teams need discipline to follow their process consistently and produce the required artifacts. Testing activities contribute much to ensure compliance. As mentioned earlier in this chapter, automated tests are a great way to create consistent test results and living documentation. Compliance is part of Release Done, and therefore part of the product, not an exam to pass at the end.

Janet gets asked one question fairly often: “Are there any domains that shouldn’t attempt to use agile methods, like regulatory?”

Is Agile Possible in a Regulated Environment?

Griffin Jones, *an agile tester who provides consulting on regulatory compliance, gets asked this question anytime someone moving to agile realizes he is an expert in testing in regulatory environments. Here's how Griffin answers.*

For me the question is personal. One of my roles is to represent the organization to external authorities that may be unfamiliar with agile. I usually start with an acknowledgment that we are regulated, what that means to the organization, and how we have implemented compliance in the context of an agile software development and testing process. Getting the team's thinking clear on the topic is critical.

First, the team has to recognize that it is simultaneously creating two things: the product *and* the regulatory documentation.

Second, the behaviors, tasks, and artifacts of the project have to address both the product needs and the regulatory needs at the same time. They are part of the same flow of actions. You can't complete one and then try to bolt the other on top. They have to be different faces of the same coin.

Third, be context sensitive and create reasonable flexibility in processes to allow quick interactive learning. The control of processes and procedures needs to be "*Auftragstaktik*," or mission-focused. Try to minimize the amount of additional work to create the artifacts to tell and substantiate the project's regulatory story.

Fourth, if you are honestly doing good competent engineering work, how can you naturally capture evidence to support the regulatory story, without breaking the flow of the work and creating inefficiency and ineffectiveness?

For me, it is about taking responsibility and preparing the project to survive the review and criticism of an external authority. Throughout each workday I ask three questions:

- Would the stakeholders be happy if they saw and heard what we are doing right now?
- Is it important that the project be able to share this information with stakeholders in the future?
- What would be the most effective and efficient way for me to memorialize this so that I can access and share it in the future, without breaking the flow?

By focusing on these key points and questions, the project is able to do regulatory-compliant agile software development. The specific project "hows" become just context-specific details.